

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS ARAPIRACA

UNIDADE EDUCACIONAL PENEDO

CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO

6º PERÍODO

BRUNO HOLLYVER ALVES DOS MARTIRES

YASMIN GOMES VENTURA

BANCO DE DADOS 2 - RELATÓRIO DE ATIVIDADES REALIZADAS

Penedo-AL

2026

BRUNO HOLLYVER ALVES DOS MARTIRES
YASMIN GOMES VENTURA

BANCO DE DADOS 2 - RELATÓRIO DE ATIVIDADES REALIZADAS

Trabalho solicitado para obtenção de nota na disciplina de BANCO DE DADOS 2 em Sistemas de Informação da Universidade Federal de Alagoas, *Campus* Arapiraca - Unidade Educacional Penedo.

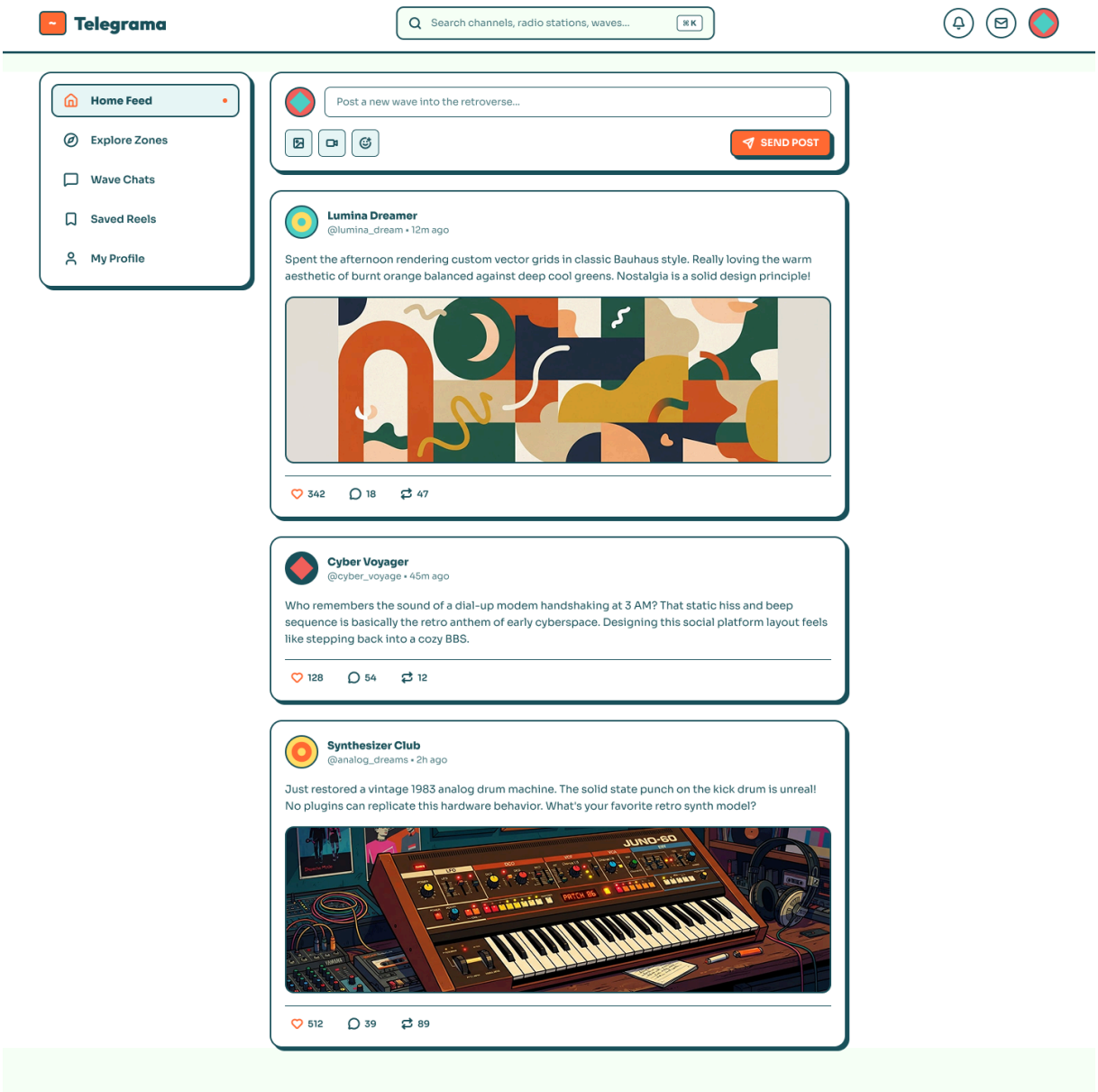
Docentes: Dr. Jario José dos Santos Júnior

Penedo/AL
2026

1. Introdução e Visão Geral

O projeto **Telegrama** nasce como uma resposta contrária à hiperestimulação visual e algorítmica das grandes redes sociais tradicionais. Inspirado na estética retrô/vintage do início dos anos 2000 e em uma paleta de cores neutra (evitando vieses de gênero como azul/rosa), o design foca em formas suaves e arredondadas para promover uma experiência de uso mais calma e menos exaustiva visualmente.

Figura 1: Feed do Telegrama



Fonte: autores

Embora a interface seja minimalista, a engenharia de dados precisa suportar a complexidade de uma rede social moderna em tempo real, garantindo consistência em relações sociais, flexibilidade de conteúdo e alta performance na entrega de feeds

2. Requisitos e Comportamento do Sistema

2.1. Requisitos Funcionais e Não Funcionais

2.1.1 Requisitos Funcionais

O sistema foi idealizado para atender às principais interações de uma rede social moderna, garantindo a integração entre os módulos relacionais e não-relacionais.

- **RF 01 - Cadastro de Usuários:** O sistema deve permitir o registro de novos usuários com dados cadastrais únicos, persistindo as informações de forma segura no banco relacional.
- **RF 02 - Serviço de Autenticação:** O sistema deve prover mecanismos de login e validação de sessão para autenticação de usuários nas rotas protegidas.
- RF03 - Publicação de Posts: O sistema deve permitir que usuários autenticados criem e publiquem postagens em texto, armazenando os dados dinamicamente no banco não-relacional.
- **RF 04 - Feed de Posts:** O sistema deve exibir um feed de publicações personalizado e paginado para o usuário, utilizando camadas de cache em memória para alta performance de leitura.
- **RF 05 - Comentários em Posts:** O sistema deve permitir que usuários interajam com publicações através do envio de comentários textuais vinculados aos posts.
- **RF 06 - Curtidas (Likes):** O sistema deve registrar reações de curtida em postagens, garantindo a contabilização e a integridade do engajamento.
- **RF 07 - Serviço de Pesquisa:** O sistema deve permitir a busca de usuários por nome ou identificador na base de dados.
- **RF 08 - Perfis de Usuário:** O sistema deve exibir a página de perfil do usuário, contendo suas informações cadastrais, histórico de posts publicados e contagem de seguidores.

Enquanto os requisitos voltados a vínculos, cadastros e integridade (`RF01`, `RF02`, `RF06`, `RF08`) são direcionados ao banco relacional (PostgreSQL), os dados dinâmicos de conteúdo e interações textuais complexas (`RF03`, `RF05`, `RF09`) aproveitam a flexibilidade do modelo orientado a documentos (MongoDB), sendo o fluxo de entrega de feed (`RF04`) otimizado em memória via Redis.

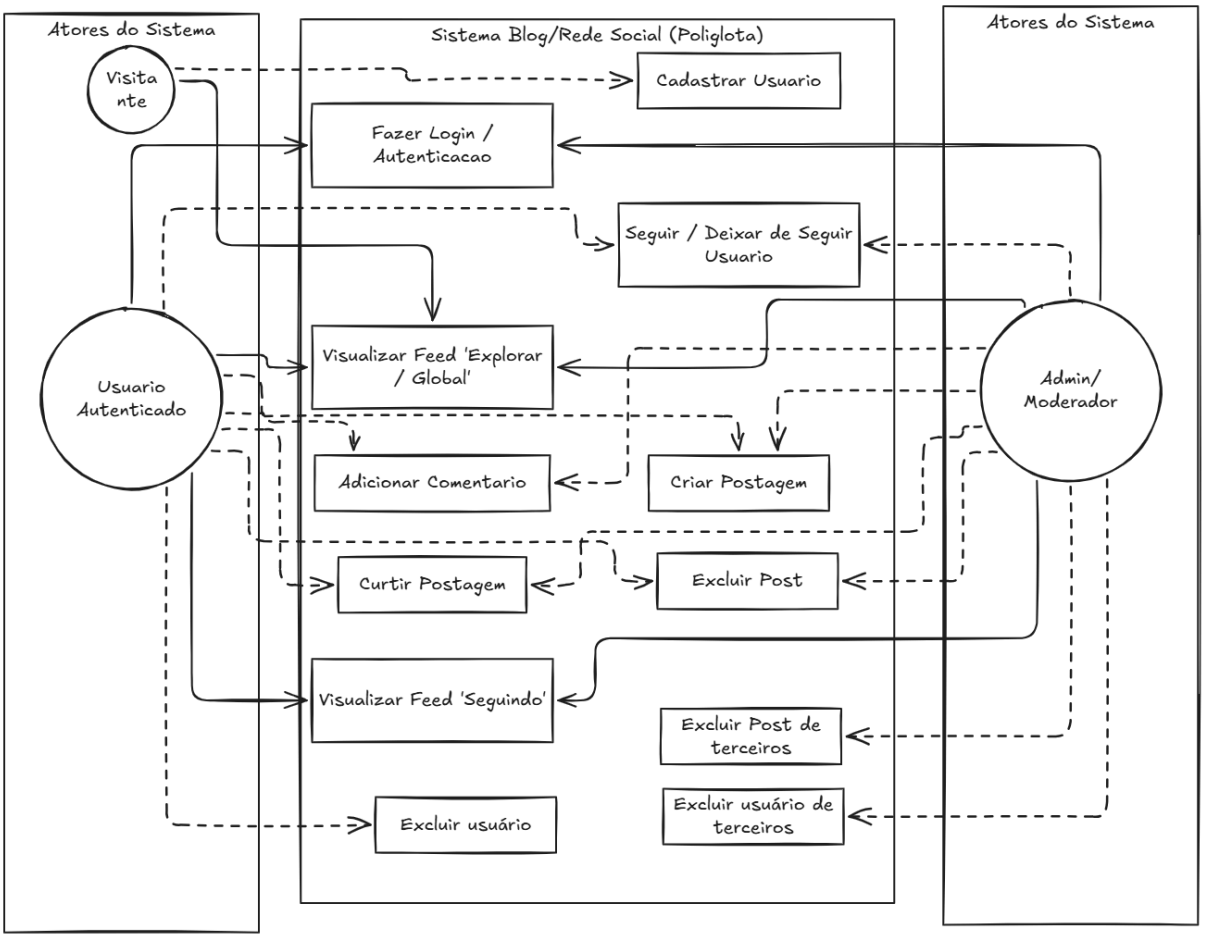
2.1.2 Não Funcionais

- **RNF 01 - Desempenho do Feed (Tempo de Resposta):** O carregamento da primeira página do feed de postagens (RF04) utilizando o cache em memória (Redis) deve ocorrer em um tempo inferior a 500 milissegundos sob condições normais de uso.

- **RNF 02 - Segurança de Acesso:** As senhas dos usuários devem ser armazenadas de forma criptografada (utilizando algoritmos de hash seguros como bcrypt), garantindo que credenciais em texto plano nunca fiquem expostas na base de dados relacional (PostgreSQL).
- **RNF 03 - Portabilidade e Implantação:** O sistema completo (API, PostgreSQL, MongoDB e Redis) deve ser empacotado e executado de maneira isolada por meio do Docker Compose, permitindo que a aplicação suba em qualquer ambiente compatível com um único comando.
- **RNF 04 - Usabilidade e Interface:** A interface do sistema deve adotar um design minimalista, com paleta de cores neutras e formas arredondadas, priorizando a redução de estímulos visuais excessivos e garantindo uma experiência leve (*Slow Tech*) para os usuários.

2.2. Diagrama de Casos de Uso

Diagrama 1: Diagrama de Casos de Uso



Fonte: autores

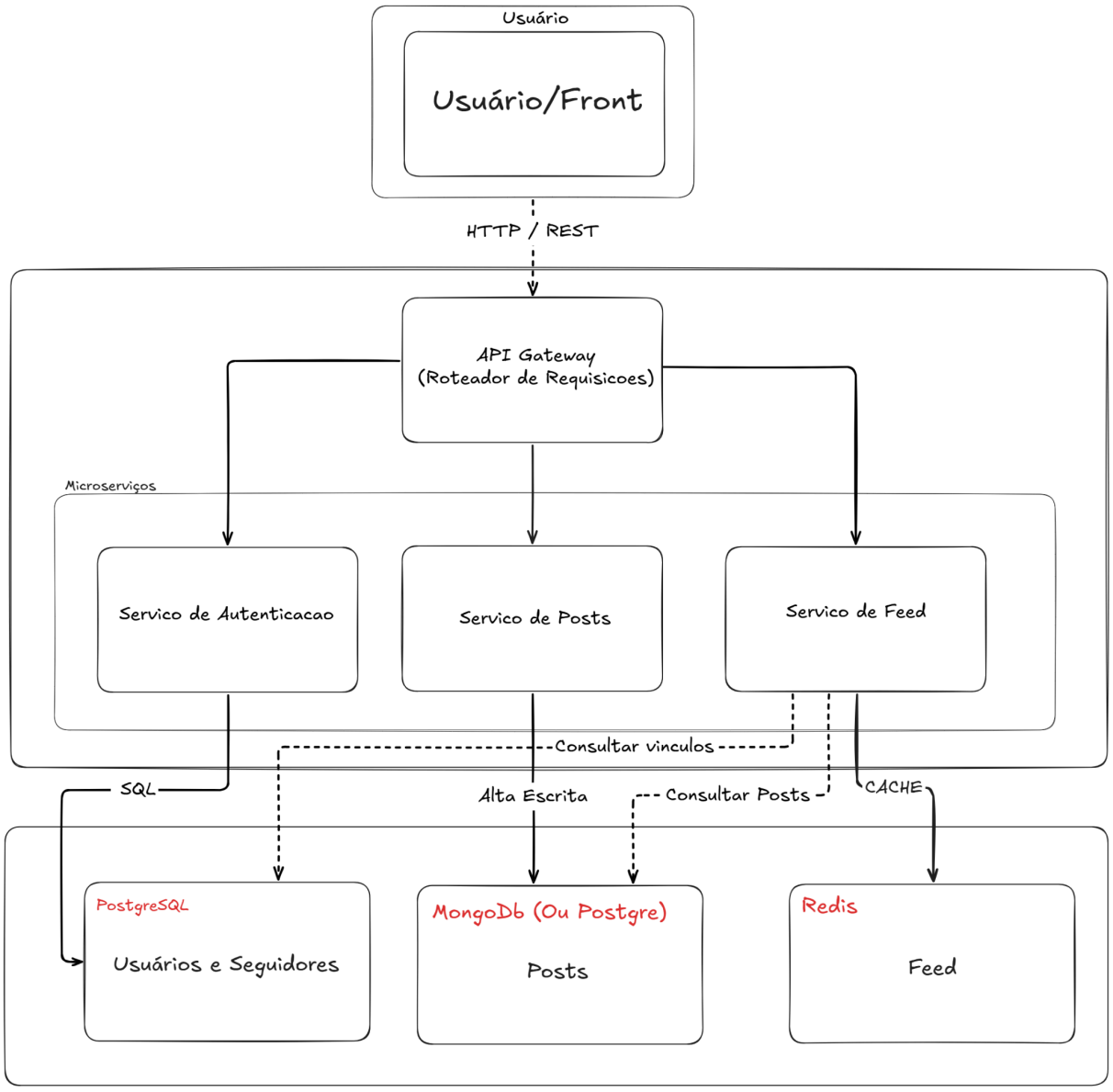
O diagrama de casos de uso modela as interações entre os atores do sistema e as funcionalidades disponíveis no Telegrama. A arquitetura de atores e as respectivas operações dividem-se da seguinte forma:

- **Atores do Sistema:**
    - **Visitante:** Usuário não autenticado que possui acesso limitado à plataforma, podendo realizar o seu cadastro ou efetuar login.
    - **Usuário Autenticado:** Ator principal que interage ativamente com o sistema após a autenticação, tendo permissão para visualizar os feeds "Explorar / Global" e "Seguindo", criar postagens, adicionar comentários, curtir postagens, seguir ou deixar de seguir outros usuários, além de gerenciar a própria conta (excluir usuário).
    - **Admin / Moderador:** Ator com privilégios elevados de controle e moderação sobre a plataforma. Além das operações comuns de autenticação e visualização de feed, possui permissões administrativas como excluir posts de terceiros, excluir usuários de terceiros e gerenciar o conteúdo geral do sistema.
- **Funcionalidades Principais:**
    - **Gerenciamento de Usuários:** Inclui Cadastrar Usuário, Fazer Login / Autenticação e Excluir usuário (bem como a exclusão de contas de terceiros pelo moderador).
    - **Interação Social:** Compreende os casos de uso Seguir / Deixar de Seguir Usuário e a visualização de conteúdos personalizados por meio dos feeds Visualizar Feed 'Explorar / Global' e Visualizar Feed 'Seguindo'.
    - **Gestão e Moderação de Conteúdo:** Engloba Criar Postagem, Adicionar Comentário, Curtir Postagem, Excluir Post próprio e Excluir Post de terceiros restrito à administração.

3. Arquitetura do Sistema e Estratégia de Persistência

3.1. Arquitetura de Microsserviços (*Database-per-Service*)

Diagrama 2: Arquitetura de Microsserviços



Fonte: autores

3.2. Justificativa da Persistência Poliglota

3.2.1 PostgreSQL (Banco Relacional - Core & Seguidores)

Operações de amizade/seguidores exigem forte integridade referencial (tabelas de relacionamento N para N, chaves estrangeiras) e garantias transacionais estritas (ACID). Se um usuário segue outro, o estado de vínculo precisa ser atômico e consistente, sem riscos de órfãos na base. O Postgres também é ideal para dados cadastrais estruturados (usuários, perfis, credenciais)

3.2.2 MongoDB (Banco NoSQL Orientado a Documentos - Postagens)

As postagens em uma rede social moderna não se limitam a texto simples; elas podem conter metadados flexíveis, imagens embedadas, enquetes, reações customizadas ou links com pré-visualizações. O esquema dinâmico (schemaless) do MongoDB permite armazenar documentos JSON complexos e aninhados sem a necessidade de dezenas de tabelas de junção (*JOINs*) caras para recuperar um único post completo.

3.2.3 Redis (Banco NoSQL Chave-Valor em Memória - Feed Paginado)

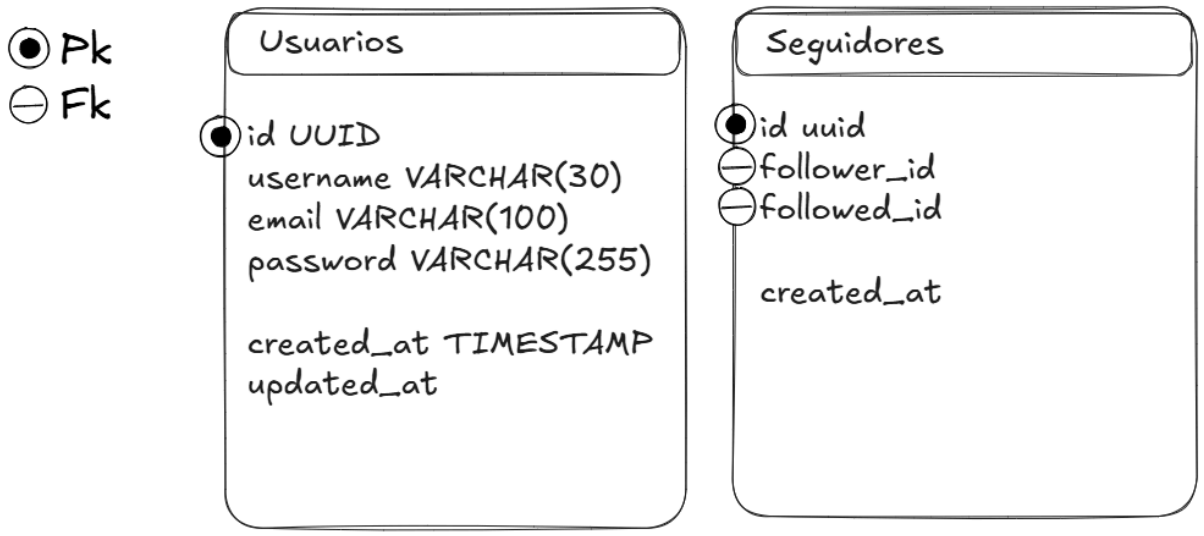
O feed é a parte mais crítica de performance em uma rede social. Consultar discos relacionais ou fazer agregações pesadas em tempo de execução para montar o feed de cada usuário causaria latência inaceitável (gargalo de consultas). Utilizando estruturas em memória do Redis (como *Sorted Sets* ou listas ordenadas por timestamp), o sistema consegue entregar o feed de maneira paginada e extremamente rápida ("carregar mais posts") sob demanda, mitigando a sobrecarga de leitura nos bancos principais.

4. Modelagem dos Dados

A modelagem de dados apresentada nesta seção possui caráter **parcial e preliminar**. Devido à complexidade e à escopo dinâmico do projeto, esta estrutura inicial serve como base para a validação dos fluxos principais, estando sujeita a alterações, refinamentos e normalizações adicionais no decorrer do desenvolvimento para garantir o pleno atendimento de todos os requisitos funcionais e não funcionais planejados.

4.1. Modelagem Relacional (SQL / PostgreSQL)

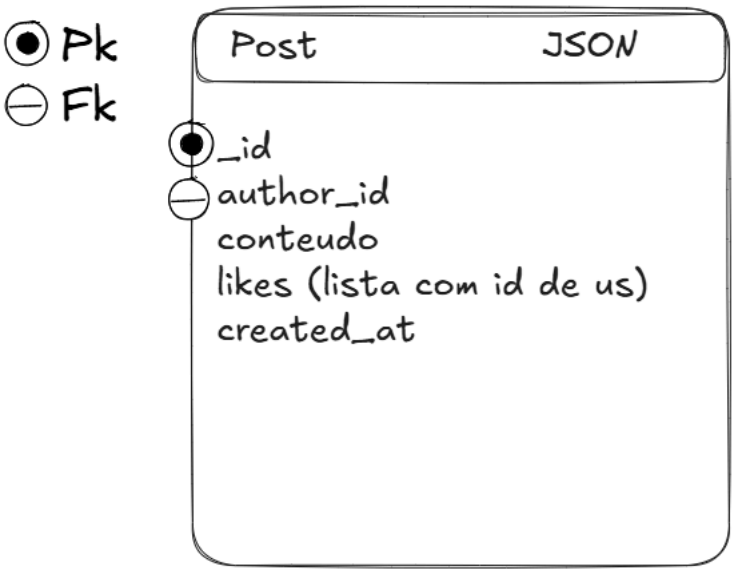
Diagrama 3: Modelagem relacional



Fonte: autores

4.2. Modelagem Não-Relacional (NoSQL / MongoDB)

Diagrama 4: Modelagem não relacional

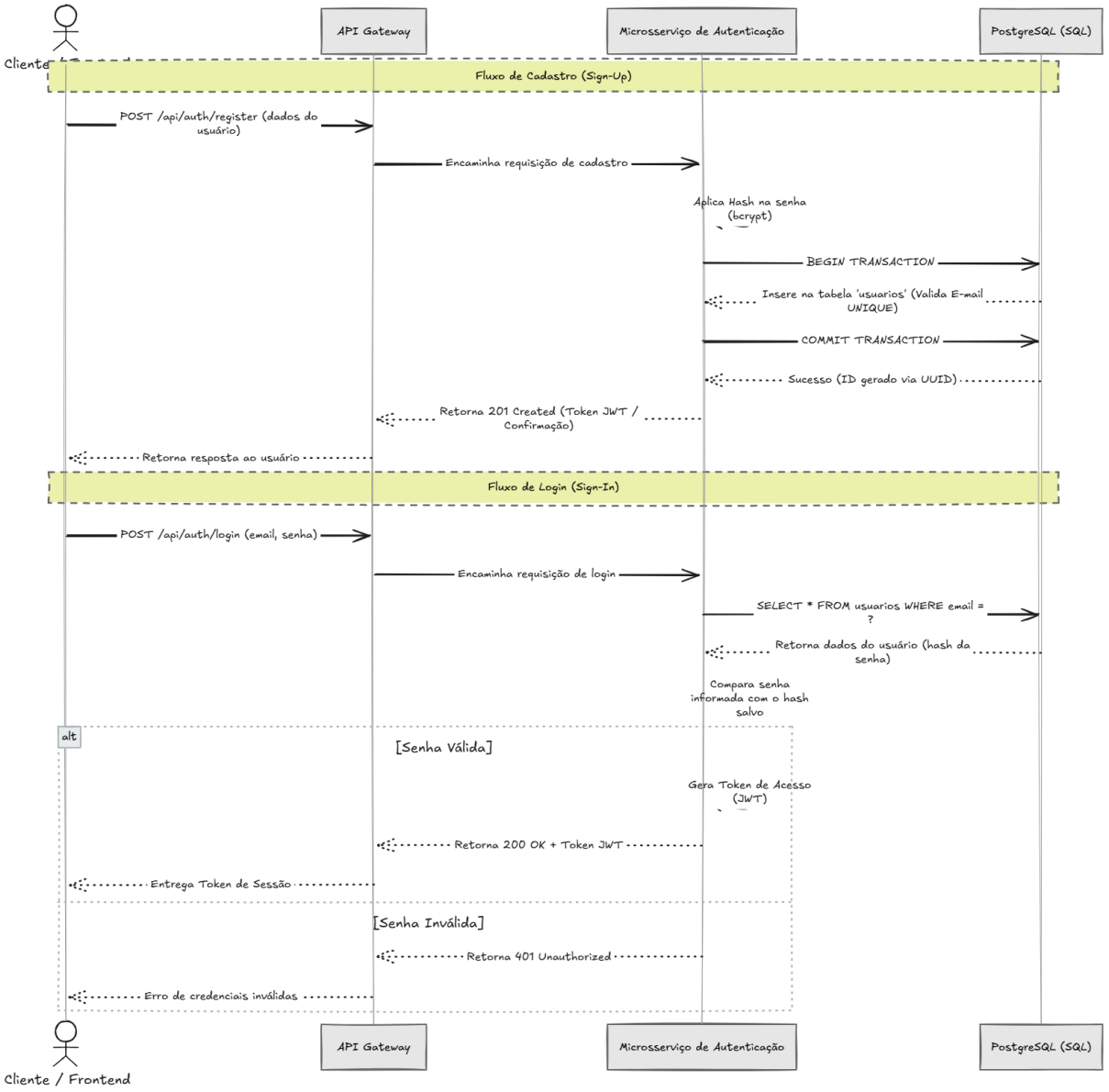


Fonte: autores

5. Fluxos Operacionais e de Integração

5.1. Fluxo de Autenticação e Transações ACID (BEGIN/COMMIT)

Diagrama 5: Diagrama de Sequência - Autenticação



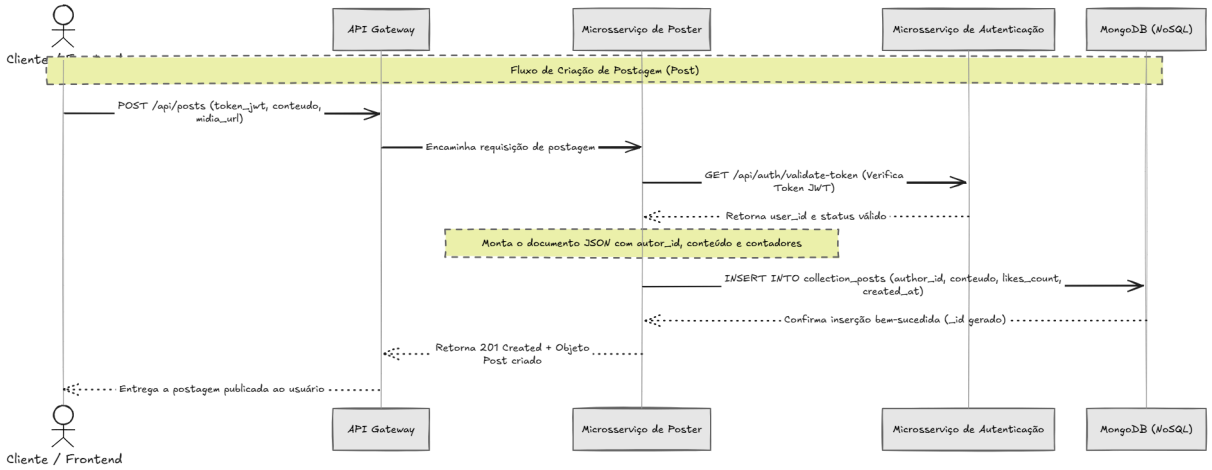
Fonte: autores

O Diagrama 5 detalha os fluxos operacionais de cadastro e de login dos usuários, destacando a garantia de atomicidade nas operações transacionais. No fluxo de cadastro, os dados enviados pelo cliente passam pelo API Gateway e chegam ao Microsserviço de Autenticação, onde a senha do usuário recebe o tratamento de hash seguro utilizando o algoritmo bcrypt. Em seguida, o sistema inicia uma transação no PostgreSQL (BEGIN TRANSACTION), executando a inserção na tabela de usuários e validando se o e-mail é

único, caso bem-sucedida, a transação é efetivada (COMMIT TRANSACTION) e um identificador único (UUID) é gerado, retornando o token JWT de confirmação ao cliente. No fluxo de login, o microsserviço valida as credenciais consultando a base relacional, compara a senha informada com o hash armazenado e, mediante validação bem-sucedida, emite o token de acesso para autenticação nas rotas protegidas do sistema.

5.2. Fluxo de Criação de Postagens

Diagrama 6: Diagrama de Sequência - Post

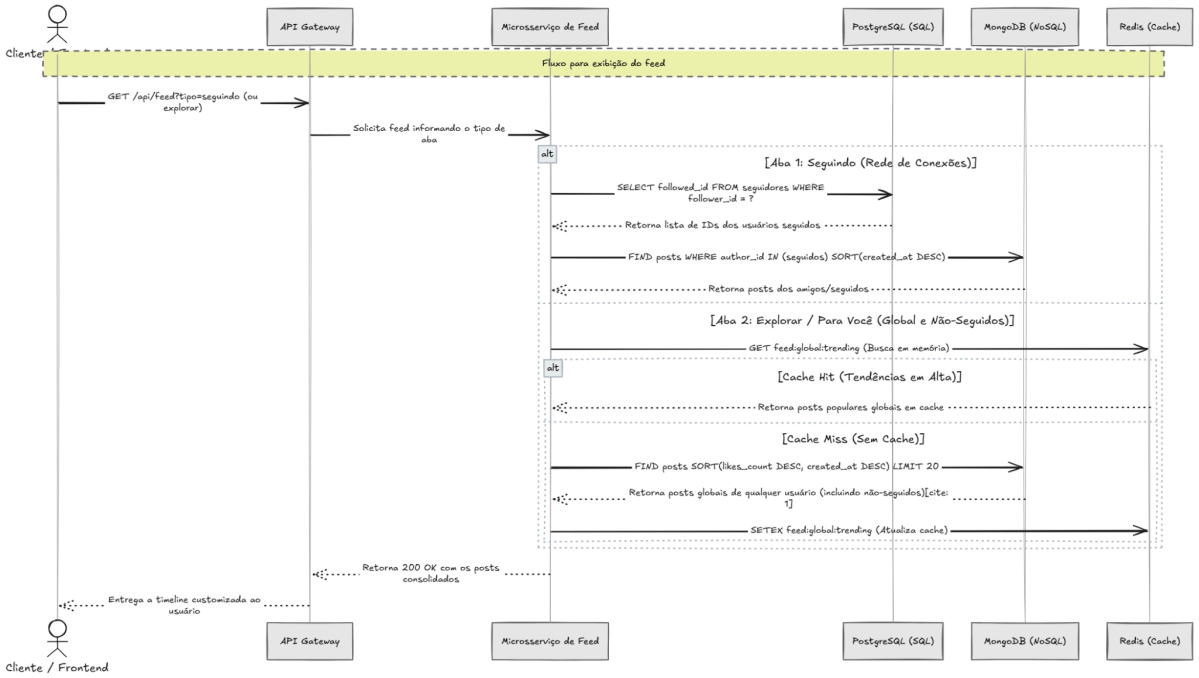


Fonte: autores

O Diagrama 6 ilustra o fluxo operacional para a criação e publicação de novas postagens na plataforma. A requisição é enviada pelo cliente por meio do protocolo HTTP através do API Gateway, que a direciona ao Microsserviço de Posts. Inicialmente, o serviço valida a autenticidade do usuário solicitante requisitando a verificação do token JWT junto ao Microsserviço de Autenticação. Após a confirmação da identidade, o microsserviço formata o documento JSON contendo o identificador do autor (author_id), o conteúdo textual e os contadores iniciais. Por fim, a postagem é inserida de forma otimizada na coleção de posts do MongoDB, retornando o status de sucesso (201 Created) e entregando o objeto publicado de volta ao frontend.

5.3. Fluxo de Montagem e Paginação de Feed

Diagrama 7: Diagrama de Sequência - Feed

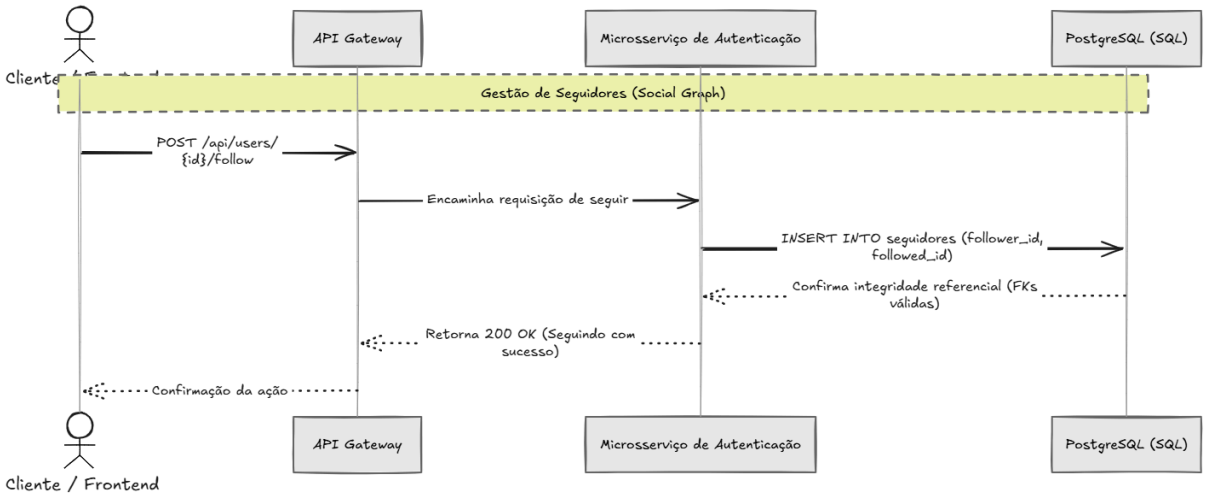


Fonte: autores

O Diagrama 7 detalha a estratégia de montagem e paginação dos feeds, dividindo o processo em cenários distintos para garantir alta performance. Quando o cliente solicita um tipo específico de feed, o API Gateway encaminha a requisição ao Microsserviço de Feed. No fluxo de postagens de conexões ('Seguindo'), o sistema consulta o banco relacional para obter a lista de IDs seguidos e busca as publicações correspondentes no MongoDB. Para o feed global ou de tendências ('Explorar / Para Você'), o sistema prioriza camadas de cache em memória no Redis (Cache Hit), evitando consultas custosas ao disco e entregando listas paginadas ordenadas por carimbo de data/hora de forma ágil.

5.4. Fluxo de Gestão de Seguidores

Diagrama 8: Diagrama de Sequência - Seguidores



Fonte: autores

O Diagrama 8 apresenta o fluxo de interação social voltado à gestão do grafo de conexões (Social Graph), especificamente a ação de seguir outro usuário. A partir da requisição POST direcionada à rota de relacionamento, o API Gateway repassa a operação ao Microsserviço de Autenticação. O microsserviço executa a persistência do vínculo inserindo os registros na tabela relacional do PostgreSQL, o qual valida automaticamente as restrições de integridade referencial por meio de chaves estrangeiras (follower_id e followed_id). Com a transação validada com sucesso, o sistema retorna o código HTTP correspondente (200 OK), confirmando a efetivação da conexão na interface do cliente.